

# Disposable Ref Structs in C# 8 with Examples

In this article, I am going to discuss **Disposable Ref Structs in C# 8** with Examples. Please read our previous article where we discussed [Static Local Functions in C# 8](#) with Examples.

## Disposable Ref Structs in C# 8:

From C# 7.2 onward, a struct can be declared with the `ref` keyword. This allows the instances of ref structs to be allocated on the stack and restricts them from moving onto the managed heap. However, this also enforces some limitations over the ref structs, the ref structs cannot implement any interface.

In C# 8.0, a special exception to the above limitation was made for the `IDisposable` interface. Ref structs can now be disposable without implementing the `IDisposable` interface, simply by having a `Dispose` method in them.

## IDisposable Interface in C#:

The Classes or Structs that implement the `IDisposable` Interface in C# are able to release their unmanaged resources. The logic for releasing the unmanaged resources is written in the `void Dispose()` function of the `IDisposable` Interface in C#. This function is called by the garbage collector on the object that it's freeing resources from. If the `IDisposable` interface is not implemented, only the managed resources will be freed by the garbage collector.

**Note:** The `Dispose` function can also be called explicitly inside the program when an object is no longer needed.

## Examples to Understand Disposable Ref Structs in C# 8:

Now, let us understand Disposable Ref Structs with examples. The Ref Structs in C# cannot implement an interface because it would expose them to boxing possibilities. That means we cannot make Ref Structs implement `IDisposable` and provide implementation to the `Dispose` method, and because of this, we cannot use Ref Structs in using statements. For a better understanding, please have a look at the below code.

```
using System;
namespace Csharp8Features
{
    public class DisposableRefStructs
    {
        public static void Main()
        {
            using (var rectangle = new Rectangle())
            {
                //Do Something
            }
        }
    }
    //Ref struct Cannot Implement the IDisposable Interface
    //hence cannot provide Implementation for Dispose method
    ref struct Rectangle : IDisposable
```

```

    {
        public void Dispose()
        {
        }
    }
}

```

The above code will give us the compilation error as **Error CS8343 'Rectangle': ref structs cannot implement interfaces**

If we remove the inheritance i.e. if we remove IDisposable Interface from the ref struct declaration, then we cannot use the ref using the using block or using declaration which is shown in the below example.

```

using System;
namespace Csharp8Features
{
    public class DisposableRefStructs
    {
        public static void Main()
        {
            using var rectangle1 = new Rectangle();
            using (var rectangle2 = new Rectangle())
            {
                //Do Something
            }
        }
    }
    //Ref struct Cannot Implement the IDisposable Interface
    //hence cannot provide Implementation for Dispose method
    ref struct Rectangle
    {
    }
}

```

When you compile the above code, you should get the following compilation error.

|   | Code   | Description                                                                                         |
|---|--------|-----------------------------------------------------------------------------------------------------|
| ✖ | CS1674 | 'Rectangle': type used in a using statement must be implicitly convertible to 'System.IDisposable'. |
| ✖ | CS1674 | 'Rectangle': type used in a using statement must be implicitly convertible to 'System.IDisposable'. |

But from C# 8 onwards, it is possible to add the Dispose method to the ref struct without implementing the IDisposable interface. So, once we add the public Dispose method to our ref struct, then this Dispose method will be called automatically by the using statement, and the whole thing compiles. For a better understanding, please have a look at the below code.

```

using System;
namespace Csharp8Features
{
    public class DisposableRefStructs

```

```

{
    public static void Main()
    {
        using var rectangle1 = new Rectangle();
        using (var rectangle2 = new Rectangle())
        {
            //Do Something
        }
    }
}
//Ref struct Cannot Implement the IDisposable Interface
//hence cannot provide Implementation for Dispose method
ref struct Rectangle
{
    public void Dispose() { }
}
}

```

Now, you can see in the above code, the ref struct Rectangle has the public Dispose method without implementing the IDisposable interface and this time we will not get any error. Even we can also use using declarations instead of using statements which is a new feature introduced in C# 8.

**Note:** If a ref struct or a read-only ref struct implements a public void Dispose() method, this would be equivalent to implementing the IDisposable interface. This means that the garbage collector would call the Dispose method when freeing resources of the said instance. Therefore, this feature allows the ref struct and read-only ref struct to be disposed of without specifying that they are implementing the IDisposable interface.

## Complete Examples to Understand Disposable Ref Structs in C# 8:

As you can see in the below example, the **ref struct Rectangle** has the public void Dispose method which allows the Rectangle to be used either in using statements or in the using declarations. In the below example, we use the new using declarations. When the main method ends the Dispose will be automatically called and release of the unmanaged resources.

```

using System;
namespace Csharp8Features
{
    public class DisposableRefStructs
    {
        public static void Main()
        {
            using var book = new Rectangle(10, 20);
            Console.WriteLine($"Area of Rectangle : {book.GetArea()}");
            Console.WriteLine("Main Method End");
        }
    }
    ref struct Rectangle
    {
        private double Height { get; set; }
    }
}

```

```

    private double Width { get; set; }
    public Rectangle(double height, double width)
    {
        Height = height;
        Width = width;
    }
    public double GetArea()
    {
        return Height * Width;
    }
    public void Dispose()
    {
        Console.WriteLine("Rectangle Object Disposed Of");
    }
}
}

```

**Output:**

```

Area of Rectangle : 200
Main Method End
Rectangle Object Disposed Of

```

In the next article, I am going to discuss [Nullable Reference Types in C# 8](#) with Examples. Here, in this article, I try to explain **Disposable Ref Structs in C# 8** with Examples. I hope you enjoy this Disposable Ref Structs in C# with Examples article.